The Random Forest Algorithm 🌳🌳🌳

The **Random Forest** is a powerful **tree ensemble** algorithm. It addresses the main weakness of a single decision tree (high sensitivity to small data changes) by building *many* different trees and combining their predictions through a voting process.

This method is significantly more robust and accurate than a single tree.

Step 1: Creating the Ensemble (Bagging)

The first key idea is to create many different trees by training them on slightly different versions of the data. This process is called **bagging** (which stands for **B**ootstrap **Agg**regating).

1. Start with your training set of size M.
2. Set the number of trees to build, B (e.g., $B = 100$).
3. **Loop B times:**
   a. Create a new, randomized training set (of size M) from the original data using **sampling with replacement**.
   b. **Train a new decision tree** on this new, sampled dataset.
4. This process results in an "ensemble" of B different decision trees.

Note: Setting B to a larger value (e.g., 64 or 128) generally improves performance, but with diminishing returns. Setting it *too* high (e.g., 1000) will just slow down computation without a meaningful increase in accuracy.

Step 2: The "Random" in Random Forest (Feature Randomization)

The bagging method alone produces "bagged decision trees." A Random Forest adds one more crucial step to **increase the diversity** of the trees.

- **Problem:** Even with different training sets, if one feature is very strong (has the highest information gain), *all* the trees might still choose to split on that same feature at the root. This would make the trees very similar and reduce the benefit of voting.
- **Solution (Feature Randomization):**
  When training each tree, at **every node** that is being split:
  1. Do not consider all N available features.
  2. Instead, pick a **random subset of K features** (where $K < N$).
  3. Find the feature with the highest information gain *only from that random subset* and use it to make the split.
- **Rule of Thumb:** A typical choice for K is $K \approx \sqrt{N}$ (the square root of the total number of

features).

This process forces each tree to be fundamentally different, as they are not all allowed to use the same "best" features, making the final ensemble much more powerful.

How a Random Forest Makes Predictions 🗳️

Once the ensemble is built, making a new prediction is done by **majority vote**:

1. Take the new test example and feed it into all B trees in the forest.
2. Each of the B trees makes its own prediction (e.g., "Cat" or "Not Cat").
3. The final prediction from the Random Forest is the class that receives the **most votes**.

This averaging process makes the final prediction very robust and insensitive to the errors or quirks of any single tree.

> ***Fun Fact:***
>
> *Q: Where does a machine learning engineer go camping?*
> *A: In a random forest!*